

Project Title: SLAIT – Secure Language Assembly Inspector Tool

Team Members:

- Maria Linkins-Nielsen – mlinkinsniel2022@my.fit.edu
- Michael Bratcher – mbratcher2021@my.fit.edu

Faculty Advisor: Dr. Marius Silaghi – msilaghi@fit.edu

Client: Dr. Marius Silaghi – CSE Professor & Faculty Advisor

Progress of Current Milestone:

Task	Completion%	Michael	Maria	Reasoning/To-Do
1. Frontend Output Widget	85%	100%	0%	The output widget has been created, and its primary purpose displays with it. This includes the basic text data organized line by line, along with dynamic graphs with multiple viewmodes and toggelable options for viewing, with there being separate graphs for flags and registers. Importing from the backend still needs to be tested, which is the major thing preventing this topic from being fully finished, and layout of how the graphs displayed may be changed, along with other mostly aesthetic low-priority items.
2. Frontend Templates	99%	100%	0%	Templates have been added to the frontend in the form of a drop down selection menu that fills the text box with pre-defined .txt files imported via json. The reasoning for

				99% completion is while the feature fully works as intended, with a default and a basic template, further templates are required to fulfill its intended purpose.
3.Implement Backend API & File Upload	%75	25%	75%	The backend execution workflow has been implemented and tested independently through a command-line interface. A sandbox job runner script (run_sandbox_job.py) now handles container creation, secure file transfer into the container, execution of the Docker-based assembly and debugging pipeline, and retrieval of structured output. The system currently produces structured JSON output including: - Program stdout - Multi-line breakpoint tracking - Register values in multiple representations (hex, unsigned, signed, byte-level, ASCII view) While the backend execution and JSON formatting are functional, full integration with the frontend HTTP POST request has not yet been completed. The remaining work involves connecting the

				frontend submission mechanism directly to the backend runner and validating the file upload and response pipeline over HTTP rather than through local CLI testing.
4. Finalize Backend Docker Execution & Register Tracking Pipeline	%95	0%	100%	The Docker-based sandbox execution pipeline has been implemented and successfully tested. The system now: - Creates a temporary Docker container per execution - Copies user-provided .asm and lines.txt files into the container - Assembles and links the program using NASM and GCC - Executes the binary inside the container - Runs GDB in batch mode with dynamically generated breakpoint scripts - Extracts the register state at multiple specified breakpoints - Returns structured JSON output to the host process The register parsing logic has been enhanced to support multiple breakpoints and multiple register representations, allowing for

				more flexible frontend visualization. Remaining work includes minor cleanup, final error-handling standardization, and direct API integration with the frontend submission system. The core execution and tracking pipeline is functionally complete.
--	--	--	--	---

Discussion of Milestone Tasks:

1.

The frontend output widget was the most complex part of the frontend development to date, as compared to previous widgets, its feature pool is much larger. The widget includes two graphs and a table, with the table simply displaying the returned JSON formatted for easier viewing. With the charts, there are two separate chart types: the register chart, and the flag chart. They are both functionally the same, and while they could technically be combined, viewing the values of the flags within the midsts of the registers is a challenge if you are tracking multiple flags and registers at the same time, even if you can hide specific registers, since flags are booleans. Each chart contains a legend for each active register, and each register has a display toggle, which can be changed on and off at any time and does not require the user to reload the chart in any way. There are also buttons to display or clear all registers to improve user experience, as toggling on and off every register can be tedious and take a bit. Finally, each chart has two modes, a standard raw value, which tracks the registers value per line, and a change over time (Δ) mode, which is a boolean chart tracking if a register has changed value since last observed.

2.

Templates, in contrast to the output widget, were the simplest addition to the frontend development. The templates are stored as .txt files and cataloged in a JSON file, meaning adding one is a 3 minute process, where the longest step would be pushing to prod.

3.

The backend execution system was significantly expanded during this milestone. Initially, individual components such as the assembly script, GDB parsing logic, and Docker container were tested independently. These components have now been integrated into a single automated pipeline that runs entirely inside a Docker container and returns structured JSON output.

A major improvement during this milestone was transitioning from single-line breakpoint tracking to multi-line breakpoint support. This required dynamically generating GDB scripts based on user input and restructuring the parsing logic to correctly group register values by breakpoint.

The backend currently supports multiple register representations, including hexadecimal, signed and unsigned integer formats, and byte-level views. This provides flexibility for frontend visualization and future expansion without requiring changes to the core execution engine.

The remaining work for this task involves connecting the frontend HTTP request to this backend execution system and validating the full end-to-end submission process.

4.

The Docker execution model has been finalized in terms of architecture. Each user request results in:

- Creation of an isolated container instance
- Copying user input files into the container
- Execution of the assembly, linking, and debugging pipeline
- Extraction of execution output and register data
- Removal of the container after completion

This design ensures that user-submitted code runs securely in isolation and does not persist on the host system. The backend now supports structured JSON output suitable for direct API responses.

The remaining improvements are primarily focused on cleanup, structured error handling, and integration with the frontend API layer rather than core execution functionality.

Contribution to Milestone Tasks:

Michael:

I worked on all of the Frontend Output Widget and the Frontend Templates.

Maria:

I focused on the backend execution environment. During this milestone, I:

- Integrated assembly, linking, and execution, and register tracking into a unified Docker based pipeline
- Implemented multi-line breakpoint support
- Developed structure JSON output for frontend consumption
- Designed a secure container lifecycle model (create, execute, extract, destroy)
- Ensured the backend execution can run fully isolated from host system

The backend pipeline is now fully functionally complete and ready for direct API integration with the frontend submission system.

1. Plan for the next Milestone (task matrix) or
[skip if this is for Milestone 6]

Task	Michael	Maria
Main: Connect Frontend HTTP Post to Backend Execution	50%	50%
End-to-End Testing (Frontend -> Backend -> Visualization)	50%	50%
Structured Error Handling & User Feedback	25%	75%

System Documentation & Demo Preparation	50%	50%
---	-----	-----

2. Discussion of each planned task for the next Milestone

- 1. Connect Frontend HTTP Post to Backend Execution
 - i. This task will integrate the frontend submission flow with the backend sandbox execution pipeline. The backend currently runs successfully via command-line testing, but this task will enable execution to be triggered through an HTTP POST request from the frontend, passing assembly code and configuration data and returning structured JSON results.
- 2. End-to-End Testing
 - i. This task will validate the full system workflow by testing real user submissions through the frontend, backend execution in Docker, and visualization of results. Testing will ensure correct data transfer, execution behavior, and compatibility between backend output and frontend visualization components.
- 3. Structured Error Handling & User Feedback
 - i. This task will improve robustness by standardizing backend error responses and exposing them to the frontend in a structured format. This will allow the frontend to display meaningful feedback to users when execution or input errors occur, improving usability and system reliability.
- 4. System Documentation & Demo Preparation
 - i. This task will focus on documenting the system architecture, execution flow, and API interfaces, as well as preparing a clear demo for evaluation. The goal is to ensure the system is understandable, presentable, and ready for the Senior Design Showcase.

3. Date(s) of meeting(s) with Client during the current milestone: n/a

4. Client feedback on the current milestone

- see Faculty Advisor Feedback below

5. Date(s) of meeting(s) with Faculty Advisor during the current milestone:

6. Faculty Advisor feedback on each task for the current Milestone

- Task 1:

- Task 2:

○ Task 3:

○ Task 4:

○ Task 5:

7. Faculty Advisor Signature: _____ Date: _____

8. Evaluation by Faculty Advisor

- Faculty Advisor: detach and return this page to Dr. Chan (HC 209) or email the scores to pkc@cs.fit.edu
- Score (0-10) for each member: circle a score (or circle two adjacent scores for .25 or write down a real number between 0 and 10)

Michael	0	1	2	3	4	5	5.5	6	6.5	7	7.5	8	8.5	9	9.5	10
Maria	0	1	2	3	4	5	5.5	6	6.5	7	7.5	8	8.5	9	9.5	10

○ Faculty Advisor Signature: _____ Date:
